Odyssey: Local Goods, Online

Design Report

Team: Team Odyssey

Members: LuoZizhen (24107742), PangXikuan (24107749), DuanYuchen (24107753)

1 Project Idea

Situation and problem. A small retailer selling clothing and household goods needs to connect browsing, stock availability and order fulfilment. When enquiries and purchase requests are handled separately, a customer may request an unavailable size, repeat delivery details or contact staff without an identifiable order. Staff must reconstruct the request, check availability and explain progress. The proposed Odyssey web application keeps these activities together through shared account, product and order records.

Users and responsibilities. The application distinguishes three signed-in roles, plus visitors. Visitors can explore products, filter results and read product details, but must sign in before adding items to a cart.

- **Customers** select clothing sizes and colours, manage their own cart and addresses, place orders, follow progress and confirm delivery receipt. They can contact staff about their own purchases and view their own membership and stored balance.
- **Staff** prepare orders, record dispatch or collection readiness and handle customer conversations. They need an actionable order list and a support inbox highlighting unanswered enquiries. They cannot change account roles or adjust stored balances.
- **Administrators** review registered accounts, assign roles, manage orders and adjust stored balances with a reason. They do not access customer-service conversations. Separating support from account administration gives each role a clear operational purpose.

Value and coverage. Customers receive a consistent route from choosing a product to receiving it. Staff act on the same order the customer sees, rather than copying information between messages. Account ownership prevents another customer from viewing or changing that order. Variant-level stock checks reduce conflicting requests for the same size and colour. Saved addresses reduce repeated entry, while linked-order support makes enquiries easier to understand.

Feasible scope. This is a single-store teaching application for a three-person team, covering clothing and household products, delivery and store collection, and English/Chinese interfaces. The central outcome is a completed or cancelled order with consistent inventory, not a payment transaction. Product photographs, prices and stock are demonstration data. Online card payments, withdrawal of stored funds, courier tracking integration and multi-vendor trading are outside the current scope. The design below reflects the current prototype and its intended boundaries; optional features are distinguished from the essential shopping workflow.

2 Main Functions

Essential functions and priorities. The first priority is a reliable customer–staff order cycle. Account control, a usable catalogue and correct inventory come first. Linked-order support is core because it resolves purchase enquiries using the same records; membership and stored balance are optional because orders can complete without them.

1. **Accounts and access.** Customers register and sign in. An authenticated customer can only use their own cart, addresses and orders. Administrators search the account list and change a user's role; the new role applies on the next request, including requests from an existing session. An administrator cannot change or delete their own account. Deletion disables access while retaining historical records, and requires the stored balance to be settled first.
2. **Product discovery and selection.** Visitors and customers browse clothing and household items, search partial words in names/descriptions, select a price range or custom bounds, and sort results. Clothing customers select an available size/colour combination. The result is a specific purchasable variant, not an ambiguous product name. Clearing a search immediately refreshes its results while retaining independent filters.
3. **Cart and checkout.** Signed-in customers add, update or remove selections. Different variants appear as separate lines; carts persist by account. Checkout offers delivery or collection, with a saved address or manual entry for delivery. The server recalculates prices, discounts and availability before saving the order. Cart contents do not reserve stock.
4. **Order fulfilment.** Customers see item specifications, delivery details and totals. The owning customer may change fulfilment method/address or cancel only while an order is pending; purchased items cannot be edited. Cancellation restores stock. Staff and administrators advance pending orders to preparing, then shipped (delivery) or ready (collection). Only the owning customer confirms delivery receipt; staff or administrators complete ready collection orders at handover. Invalid transitions are rejected on the server.
5. **Customer support.** Customers send text, an image or a reference to their own order. Staff browse conversations and see waiting/unread indicators. Images and order references inherit conversation access checks. A staff member marks a replied-to conversation resolved; a new message reopens it. Administrators cannot use either customer or staff support routes.

Optional additions. Saved address management supports multiple entries and one default. Membership uses completed-order spending to select Essential, Silver (5% off from €200), Gold (10% from €500) or Signature (15% from €1,000); a higher tier benefits subsequent orders. The order stores the discount used at checkout. A stored-balance page shows credits/debits; administrators can adjust balances with an audit reason, but users cannot self-credit and checkout does not yet spend this balance. These additions improve convenience without becoming prerequisites for placing an order.

Usability priority. Responsive layouts, bilingual text and restrained motion support the core tasks. Reduced-motion preferences are respected. Decorative animation never delays a submit action. Separate account windows are useful for demonstrating customer and staff interactions on one browser; they are an enhancement, not a replacement for server-side permission checks.

3 Application Design

Application structure. Odyssey uses a server-rendered, layered application rather than separate microservices. Figure 1 shows how customer and staff actions reach the same order service and stored order. Controllers interpret input, services enforce rules, and JDBC persists changes. Responses return through MVC to the relevant browser view.

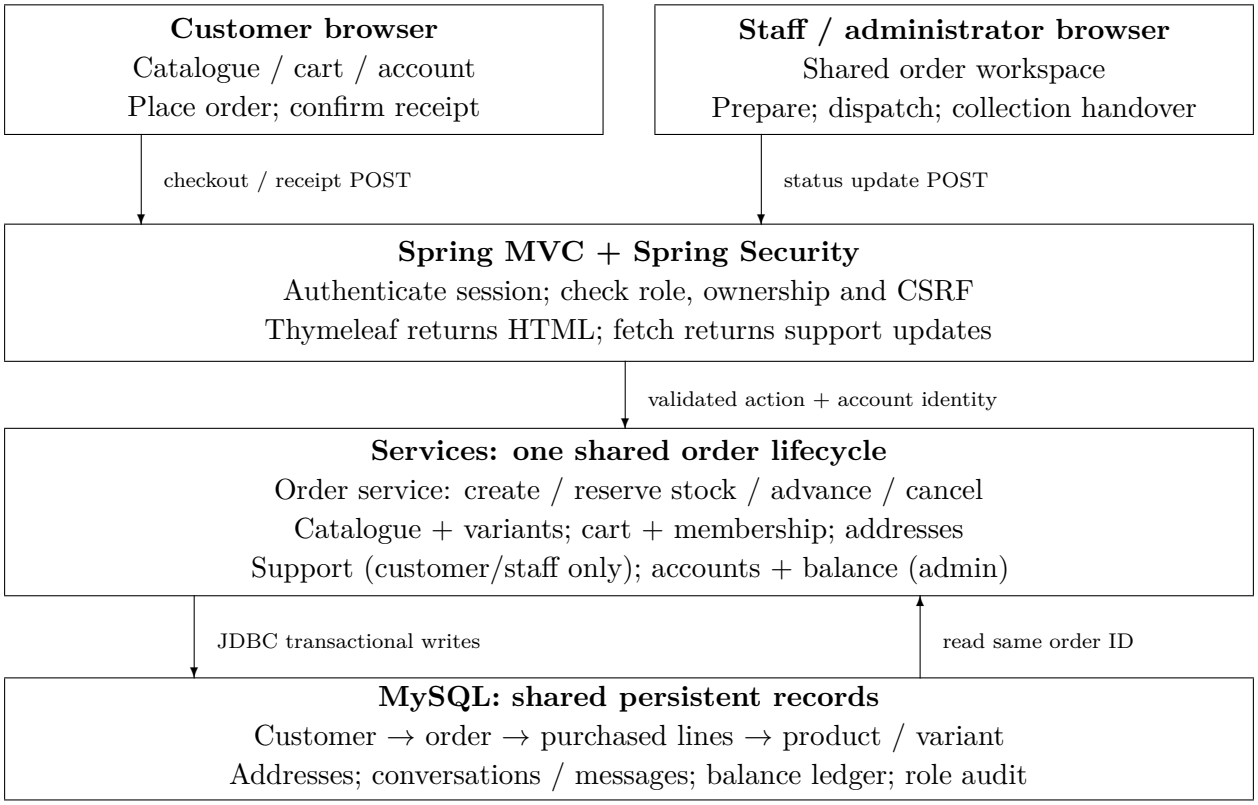

Figure 1: Customer and staff actions converge on shared order processing and records. Both browser interfaces use HTML/CSS and JavaScript.

Responsibilities and boundaries. Browser validation helps users correct inputs, but the server decides who may act and whether a change is valid. Catalogue services read products and options. Order services own inventory reservation, order creation and cancellation. Support services check conversation ownership and distinguish customer from staff activity. Account services govern role changes; the balance service records authorised adjustments. Static product images are served with the application; uploaded support images are stored with access-controlled records in MySQL.

Consistency. A checkout transaction locks the relevant account and inventory, validates every line, writes the order and its price/discount snapshots, and clears the cart. Failure rolls back the changes. A unique checkout token prevents a repeated submission from creating another order. Balance adjustments use row locks and their own request token, so concurrent deductions cannot create a negative balance and retries do not double-credit an account. Role changes record the actor and previous/new roles. These rules belong to services and database transactions, not just hidden buttons.

Data records and relationships. Table 1 identifies the main records and their links. ID labels describe conceptual keys rather than every physical column. A customer owns many orders; an order owns many lines; a line refers to one product and optionally one variant.

Record	Information stored	Functions using/updating it
Account / role audit	ID, name, email, password hash, role, creation time; actor and old/new role	Registration, login, role refresh, administrator account changes and deletion
Product / variant	Product ID, name, category, description, price, images; variant ID, product ID, size, colour, stock	Catalogue search, specification selection and inventory checks
Account cart	Account ID, product or variant ID, quantity	Cart editing; checkout reads and clears purchased lines
Order	Order ID, customer ID, checkout token, status, fulfilment, copied address, total; tier and discount snapshot	Checkout, status updates, cancellation, receipt and membership calculation
Order line	Order ID, product ID, optional variant ID; copied name, unit price, quantity, size/colour	Checkout writes purchase snapshot; order details display it; cancellation releases stock
Saved address	Owner, recipient, phone, full address, default flag	Address management and checkout selection
Support conversation / message	Conversation ID, customer ID; message ID, conversation ID, sender, text, time; read/resolved markers; optional image or order ID	Customer enquiries, employee reply queue, unread tracking and resolution
Stored-balance entry	Owner, administrator actor, amount, resulting balance, reason, timestamp, request token	Balance display and audited administrator credits/debits

Table 1: Persistent records and the functions that depend on them.

Relationship rules. The account–cart link isolates shopping across customers. A product has several size/colour variants, each with its own stock; household products can use product-level stock. An order's copied address is independent of later address-book edits. Price and specification snapshots preserve historical meaning when a catalogue entry changes. Conversations belong to customers; an attached order must belong to that conversation's customer. Read markers are per viewer, so "unread" is distinct from "waiting for a reply". Stored balance is derived from an append-only ledger and does not contribute to membership spending.

Deletion and retained history. Logical account deletion disables access while preserving order, support and ledger links. The retained email cannot be registered again. Historical snapshots remain readable to authorised roles; this is retention for traceability, not complete personal-data erasure.

User workflow: a delivery purchase.

1. A customer signs in, finds a shirt, selects a size/colour and adds it to the cart. Cart processing stores that variant and quantity under the customer's account.
2. At checkout, the customer selects delivery and an address. The order service rechecks stock and prices, derives the membership tier from completed orders, and calculates the total on the server.
3. One transaction reserves stock, saves the pending order, item and discount snapshots, copies the delivery address and clears the cart. The browser redirects to the customer's order list. Staff see this same pending order in their workspace.
4. Staff start preparation, then mark the delivery shipped. The customer sees the new status and confirms receipt after delivery. The order becomes completed; its discounted total then contributes to future membership eligibility. For collection, staff instead mark it ready and complete it at handover.

Exception: competing requests for the last unit. Carts do not reserve stock. Checkout locks and conditionally decrements inventory, so only a transaction with sufficient stock succeeds. A competing transaction rolls back, retains its cart and reports unavailable stock. The customer can reduce quantity or choose another option; no partial order remains. Cancelling a pending order before preparation restores its reserved stock.

4 Technology Choices

Browser. HTML5, CSS and small JavaScript modules provide responsive pages, bilingual labels, search controls, image switching and progressive animation. Thymeleaf renders the initial page on the server. This suits a small team: there is one application and no separate frontend API/authentication stack. Forms and links remain the core interaction model; JavaScript enhances search clearing and address editing rather than controlling ownership or prices.

Server. Java 17 and Spring Boot with Spring MVC organise controllers and services. Spring Security supplies password hashing (BCrypt), session authentication, route permissions and CSRF protection. Explicit ownership checks protect individual records. These choices fit the role-based shopping workflow and avoid implementing authentication primitives from scratch. Spring Session provides isolated account-window sessions in memory; this prototype is designed for a single application instance, not distributed session deployment.

Storage and build. MySQL with JDBC supports foreign keys, decimal monetary values and transactions for checkout and balance changes. SQL migration scripts make the schema reproducible; Maven manages builds and dependencies. JUnit, Spring MVC/Security tests and H2 in MySQL compatibility mode exercise permissions, rendering and failure paths. H2 is useful for isolated tests, but MySQL smoke checks remain necessary because the two engines are not identical.

Additional learning and integration. The integration plan targets framework-specific risks: first verify login, role restrictions and CSRF in a small Spring Security example; then test Spring Session window isolation and logout; finally connect support polling with ownership checks. Polling keeps infrastructure simple but introduces a short delay. MySQL tests will exercise competing stock updates and duplicate checkout tokens before optional features are integrated. This staged plan applies wherever further learning is needed, without assuming prior framework knowledge.